📘 MÓDULO RESUMEN PRÁCTICO Y DUDAS CLAVE DE EXAMEN

---

1. GITHUB ACTIONS: ESTRUCTURA COMPLETA Y SISTEMAS OPERATIVOS

📁 Ubicación exacta de la carpeta:

`.github/workflows/mi-workflow.yml`

(¡Ojo! La carpeta principal lleva un punto delante: .github y dentro va la carpeta workflows en plural).

💻 ¿Cómo se escriben exactamente los Sistemas Operativos (runs-on)?

- runs-on: ubuntu-latest ➔ Linux (el más usado y rápido).
- runs-on: windows-latest ➔ Windows (con 's' final).
- runs-on: macos-latest ➔ macOS (todo junto, sin guion).

📝 Plantilla completa y básica de examen con Steps:

```
name: Workflow de Examen


on: [push]
```

```
jobs:
  mi-trabajo:
    runs-on: ubuntu-latest # O windows-latest / macos-latest

    steps:
      # Step 1: Siempre descargar el código primero
      - name: Descargar repositorio
        uses: actions/checkout@v3

      # Step 2: Ejecutar comandos de consola
      - name: Probar ejecucion
        run: echo "Paso 1 completado"

      # Step 3: Otro paso de consola
      - name: Ver archivos del proyecto
        run: ls -la
```

2. GUÍA COMPLETA DEL .gitignore (SINTAXIS Y RUTAS)

El archivo se crea en la raíz del proyecto y se llama exactamente .gitignore.

Sintaxis y ejemplos reales:

- **Nombre exacto de archivo:** debug.log ➔ Ignora ese archivo concreto en cualquier carpeta.
- **Por extensión (Comodín *):** *.log ➔ Ignora TODOS los archivos que terminen en .log.
- **Carpeta completa (Barra al final /):** node_modules/ o dist/ ➔ Ignora la carpeta

entera y todo su contenido.

- **Ruta específica:** temp/logs/ ➔ Ignora solo la carpeta logs que esté dentro de temp.
- **Excepción (Signo !):** !importante.log ➔ Si has ignorado todos los .log, este en concreto SÍ lo rastreará.

3. GITHUB PAGES: RUTAS Y PESTAÑAS EXACTAS

1. **¿Dónde colocar los archivos web?** En la raíz de tu proyecto (/root) debe haber un archivo llamado exactamente index.html.
2. **Menú de GitHub en el navegador:**
    - Entras a tu repositorio ➔ Pestaña superior **Settings**.
    - En el menú lateral izquierdo ➔ Sección **Pages** (debajo de *Code and automation*).
    - En *Build and deployment* ➔ **Source:** Seleccionas *Deploy from a branch*.
    - En *Branch:* Seleccionas main (o master) y la carpeta / (root) ➔ Clic en **Save**.

4. PERSONAL ACCESS TOKENS (PAT): CUÁNDO, DÓNDE Y PERMISOS

¿Cuándo te va a pedir un Token?

Cuando intentes hacer un git push desde la terminal de Linux/VirtualBox hacia GitHub usando HTTPS y te pida la contraseña. GitHub **ya no acepta tu contraseña normal**, te exige usar un **Personal Access Token (PAT)** como si fuera tu clave.

¿Dónde se crea en la web de GitHub?

1. Haz clic en tu foto de perfil (arriba a la derecha) ➔ **Settings** generales de la cuenta.
2. Baja en el menú lateral izquierdo hasta **Developer settings**.
3. Selecciona **Personal access tokens ➔ Tokens (classic) ➔ Generate new token**.

🔒 ¿Qué permisos darle al Token según el uso? (Muy importante)

- **Para el examen / uso habitual (Push, Pull, Clone):** Solo necesitas marcar la casilla principal **`repo`** (Full control of private repositories). ¡NO hace falta darle todos los permisos! Con `repo` puedes hacer push, pull y gestionar el código sin dar acceso a cosas peligrosas.
- **Si vas a administrar la cuenta / workflows:** Solo en caso especial se marca también **`workflow`** (para modificar archivos YAML de GitHub Actions si el token no te dejaba).